Week 10

1. Write a Java program to demonstrate **exception handling using try and catch** by handling division by zero.

File Edit Format View Help

```
public class DivisionExample {
    public static void main(String[] args) {
        int numerator = 10;
        int denominator = 0;

        try {
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero!");
            System.out.println("Exception message: " + e.getMessage());
        } finally {
            System.out.println("Division operation attempted.");
        }

        System.out.println("Program continues after exception handling.");
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac DivisionExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java DivisionExample
Error: Cannot divide by zero!
Exception message: / by zero
Division operation attempted.
Program continues after exception handling.
```

2. Write a Java program to demonstrate different **types of exceptions** such as ArithmeticException, ArrayIndexOutOfBoundsException, and NullPointerException.

File Edit Format View Help

```
public class MultipleExceptionsExample {
    public static void main(String[] args) {

        // 1. ArithmeticException - division by zero
        try {
            int a = 10, b = 0;
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException caught: " + e.getMessage());
        }

        System.out.println("-----------------------------");

        // 2. ArrayIndexOutOfBoundsException - invalid array index
        try {
            int[] numbers = {10, 20, 30};
            System.out.println("Accessing index 5: " + numbers[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException caught: " + e.getMe
        }

        System.out.println("-----------------------------");

        // 3. NullPointerException - calling a method on a null reference
        try {
            String text = null;
            System.out.println("Length: " + text.length());
        } catch (NullPointerException e) {
            System.out.println("NullPointerException caught: " + e.getMessage());
        }

        System.out.println("-----------------------------");
        System.out.println("Program completed successfully after handling all exce
    }
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac MultipleExceptionsExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java MultipleExceptionsExample
ArithmeticException caught: / by zero
-----------------------------
ArrayIndexOutOfBoundsException caught: 5
-----------------------------
NullPointerException caught: null
-----------------------------
Program completed successfully after handling all exceptions.
```

3.Write a Java program to demonstrate an **uncaught exception** and observe the default JVM exception message and stack trace.

File Edit Format View Help

```
public class UncaughtExceptionExample {

    public static void main(String[] args) {
        System.out.println("Program started...");

        int[] numbers = {10, 20, 30};

        // Calling a method that will throw an uncaught exception
        processArray(numbers);

        // This line will never execute because the exception is not caught
        System.out.println("Program finished.");
    }

    public static void processArray(int[] arr) {
        System.out.println("Processing array...");
        divide(arr[0], 0);  // Deliberately causes ArithmeticException
    }

    public static void divide(int a, int b) {
        int result = a / b;  // No try-catch here – exception propagates up
        System.out.println("Result: " + result);
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac UncaughtExceptionExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java UncaughtExceptionExample
Program started...
Processing array...
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at UncaughtExceptionExample.divide(UncaughtExceptionExample.java:21)
        at UncaughtExceptionExample.processArray(UncaughtExceptionExample.java:17)
        at UncaughtExceptionExample.main(UncaughtExceptionExample.java:9)

C:\Users\ITLAB2\Desktop\25EU02098>
```

4.Write a Java program to handle **multiple exceptions using multiple catch clauses**.

File Edit Format View Help

```
import java.util.Scanner;

public class MultiCatchExample {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int[] numbers = {10, 20, 30};

        try {
            System.out.print("Enter an index to access (0-2): ");
            int index = scanner.nextInt();

            System.out.print("Enter a number to divide 100 by: ");
            int divisor = scanner.nextInt();

            int result = 100 / divisor;
            System.out.println("100 / " + divisor + " = " + result);

            System.out.println("Value at index " + index + ": " + numbers[index]);

        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException caught: " + e.getMessage());
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ArrayIndexOutOfBoundsException caught: " + e.getMe
        } catch (java.util.InputMismatchException e) {
            System.out.println("InputMismatchException caught: Please enter valid
        } finally {
            scanner.close();
            System.out.println("Scanner closed. Execution finished.");
        }
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac MultiCatchExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java MultiCatchExample
Enter an index to access (0-2): 1
Enter a number to divide 100 by: 500
100 / 500 = 0
Value at index 1: 20
Scanner closed. Execution finished.

C:\Users\ITLAB2\Desktop\25EU02098>
```

5.Write a Java program to demonstrate the use of the **throw statement** to explicitly generate an exception when a given condition is violated.

```
public class ThrowExample {

    public static void main(String[] args) {
        try {
            validateAge(15);
            System.out.println("Age is valid. Proceeding...");
        } catch (IllegalArgumentException e) {
            System.out.println("Caught exception: " + e.getMessage());
        }

        System.out.println("------------------------------");

        try {
            checkBankBalance(500, 1000);
            System.out.println("Withdrawal successful.");
        } catch (IllegalStateException e) {
            System.out.println("Caught exception: " + e.getMessage());
        }

        System.out.println("------------------------------");
        System.out.println("Program completed.");
    }

    // Explicitly throws an exception if age condition is violated
    public static void validateAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or above. Provided:
        }
        System.out.println("Age " + age + " accepted.");
    }

    // Explicitly throws an exception if withdrawal amount exceeds balance
    public static void checkBankBalance(int balance, int withdrawAmount) {
        if (withdrawAmount > balance) {
            throw new IllegalStateException(
                "Insufficient funds: balance is " + balance + ", attempted withdra
            );
        }
        System.out.println("Balance sufficient. New balance: " + (balance - withdr
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac ThrowExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java ThrowExample
Caught exception: Age must be 18 or above. Provided: 15
----------------------------
Caught exception: Insufficient funds: balance is 500, attempted withdrawal is 100
----------------------------
Program completed.

C:\Users\ITLAB2\Desktop\25EU02098>
```

6. Write a Java program to demonstrate the use of the **throws keyword** by propagating an exception to the calling method.

```
import java.io.IOException;

public class ThrowsExample {

    public static void main(String[] args) {
        try {
            readConfigFile("settings.txt");
            System.out.println("Config loaded successfully.");
        } catch (IOException e) {
            System.out.println("Caught exception in main: " + e.getMessage());
        }

        System.out.println("----------------------------");
        System.out.println("Program completed.");
    }

    // Declares it MIGHT throw a checked exception, but doesn't handle it itself
    public static void readConfigFile(String filename) throws IOException {
        validateFile(filename);
        System.out.println("Reading from " + filename + "...");
    }

    // This method actually throws the exception; it also propagates it upward
    public static void validateFile(String filename) throws IOException {
        if (!filename.endsWith(".cfg")) {
            throw new IOException("Invalid config file format: " + filename + " (e
        }
        System.out.println("File format validated.");
    }
}
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac ThrowsExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java ThrowsExample
Caught exception in main: Invalid config file format: settings.txt (expected .cfg)
----------------------------
Program completed.
```

7. Write a Java program to demonstrate the use of the **finally block** for executing statements regardless of whether an exception occurs.

```
public class FinallyExample {

    public static void main(String[] args) {

        // Case 1: No exception occurs
        System.out.println("Case 1: No exception");
        divide(10, 2);

        System.out.println("-----------------------------");

        // Case 2: Exception occurs and is caught
        System.out.println("Case 2: Exception caught");
        divide(10, 0);

        System.out.println("-----------------------------");

        // Case 3: finally runs even when returning from inside try/catch
        System.out.println("Case 3: finally with return statement");
        System.out.println("Returned value: " + divideAndReturn(20, 5));
    }

    public static void divide(int a, int b) {
        try {
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        } finally {
            System.out.println("Finally block executed (divide method).");
        }
    }

    public static int divideAndReturn(int a, int b) {
        try {
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac FinallyExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java FinallyExample
Case 1: No exception
Result: 5
Finally block executed (divide method).
-----------------------------
Case 2: Exception caught
Exception caught: / by zero
Finally block executed (divide method).
-----------------------------
Case 3: finally with return statement
Finally block executed (divideAndReturn method), before returning.
Returned value: 4
```

8. Write a Java program to create a **user-defined exception** by extending the Exception class.

```
// Custom checked exception - extends Exception
class InsufficientFundsException extends Exception {

    private double shortfall;

    public InsufficientFundsException(String message, double shortfall) {
        super(message);  // Pass message up to the Exception superclass
        this.shortfall = shortfall;
    }

    public double getShortfall() {
        return shortfall;
    }
}

public class CustomExceptionExample {

    public static void main(String[] args) {
        BankAccount account = new BankAccount(1000);

        try {
            System.out.println("Attempting to withdraw 1500...");
            account.withdraw(1500);
            System.out.println("Withdrawal successful.");
        } catch (InsufficientFundsException e) {
            System.out.println("Caught custom exception: " + e.getMessage());
            System.out.println("Amount short by: " + e.getShortfall());
        }

        System.out.println("----------------------------");

        try {
            System.out.println("Attempting to withdraw 300...");
            account.withdraw(300);
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac CustomExceptionExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java CustomExceptionExample
Attempting to withdraw 1500...
Caught custom exception: Cannot withdraw 1500.0; balance is only 1000.0
Amount short by: 500.0
----------------------------
Attempting to withdraw 300...
Withdrawal successful. New balance: 700.0
----------------------------
Program completed.
```

9. Write a Java program to validate a student's marks and throw a **custom exception** when the marks are outside the valid range.

```
class InvalidMarksException extends Exception {

    private int invalidValue;

    public InvalidMarksException(String message, int invalidValue) {
        super(message);
        this.invalidValue = invalidValue;
    }

    public int getInvalidValue() {
        return invalidValue;
    }
}

public class MarksValidationExample {

    public static void main(String[] args) {
        int[] marksToTest = {85, -5, 105, 42, 100, 0};

        for (int marks : marksToTest) {
            try {
                validateMarks(marks);
                System.out.println("Marks " + marks + " -> Valid. Grade: " + calcu
            } catch (InvalidMarksException e) {
                System.out.println("Marks " + marks + " -> Invalid: " + e.getMessa
            }
            System.out.println("------------------------------");
        }

        System.out.println("Validation completed for all entries.");
    }

    public static void validateMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac MarksValidationExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java MarksValidationExample
Marks 85 -> Valid. Grade: B
------------------------------
Marks -5 -> Invalid: Marks must be between 0 and 100. Provided: -5
------------------------------
Marks 105 -> Invalid: Marks must be between 0 and 100. Provided: 105
------------------------------
Marks 42 -> Valid. Grade: D
------------------------------
Marks 100 -> Valid. Grade: A
------------------------------
Marks 0 -> Valid. Grade: F
------------------------------
Validation completed for all entries.

C:\Users\ITLAB2\Desktop\25EU02098>
```

10. Write a Java program that combines **try, multiple catch, throw, throws and finally** in a single application.

```
import java.util.Scanner;

class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class AllExceptionFeaturesExample {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            System.out.print("Enter your age: ");
            int age = scanner.nextInt();

            System.out.print("Enter a number to divide 100 by: ");
            int divisor = scanner.nextInt();

            System.out.print("Enter an index to access (array has 3 elements): ");
            int index = scanner.nextInt();

            registerUser(age);

            int result = 100 / divisor;
            System.out.println("100 / " + divisor + " = " + result);

            int[] data = {10, 20, 30};
            System.out.println("Value at index " + index + ": " + data[index]);

            System.out.println("All operations completed successfully.");
```

```
C:\Users\ITLAB2\Desktop\25EU02098>javac MarksValidationExample.java

C:\Users\ITLAB2\Desktop\25EU02098>java MarksValidationExample
Marks 85 -> Valid. Grade: B
----------------------------
Marks -5 -> Invalid: Marks must be between 0 and 100. Provided: -5
----------------------------
Marks 105 -> Invalid: Marks must be between 0 and 100. Provided: 105
----------------------------
Marks 42 -> Valid. Grade: D
----------------------------
Marks 100 -> Valid. Grade: A
----------------------------
Marks 0 -> Valid. Grade: F
----------------------------
Validation completed for all entries.
```

12.Write a Java program to demonstrate the use of InputStream and OutputStream to read bytes from an input source and write bytes to an output destination.

```
import java.io.*;

public class SimpleStreamDemo {
    public static void main(String[] args) throws IOException {

        // Create a small input file to read from
        try (OutputStream out = new FileOutputStream("input.txt")) {
            out.write("Hello, Streams!".getBytes());
        }

        // Read bytes from input.txt and write them to output.txt
        try (InputStream in = new FileInputStream("input.txt");
             OutputStream out = new FileOutputStream("output.txt")) {

            int b;
            while ((b = in.read()) != -1) {   // read one byte (-1 means end of file)
                out.write(b);                 // write that byte
            }
        }

        System.out.println("Copied input.txt to output.txt");
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Sim
pleStreamDemo.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Simp
leStreamDemo
Copied input.txt to output.txt
```

13. Write a Java program to read the contents of a file byte-by-byte using FileInputStream and display the contents on the console.

```
import java.io.FileInputStream;
import java.io.IOException;

public class ReadFileByteByByte {
    public static void main(String[] args) {
        // Specify the path to the file you want to read
        String filePath = "input.txt";

        FileInputStream fis = null;

        try {
            fis = new FileInputStream(filePath);
            int byteData;

            // read() returns -1 when end of file is reached
            while ((byteData = fis.read()) != -1) {
                // Cast to char to display readable text
                System.out.print((char) byteData);
            }

        } catch (IOException e) {
            System.out.println("Error reading file: " + e.getMessage());
        } finally {
            // Always close the stream to release resources
            if (fis != null) {
                try {
                    fis.close();
                } catch (IOException e) {
                    System.out.println("Error closing file: " + e.getMessage());
                }
            }
        }
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Rea
dFileByteByByte.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java Read
FileByteByByte
Hello, Streams!
```

14. Write a Java program to copy the contents of one file to another file using FileInputStream and FileOutputStream.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class FileCopy {
    public static void main(String[] args) throws IOException {
        String sourceFile = "source.txt";
        String destFile = "destination.txt";

        FileInputStream fis = new FileInputStream(sourceFile);
        FileOutputStream fos = new FileOutputStream(destFile);

        int i;
        while ((i = fis.read()) != -1) {
            fos.write(i);
        }

        fis.close();
        fos.close();

        System.out.println("File copied successfully.");
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Fil
eCopy.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java File
Copy
File copied successfully.
```

15. Write a Java program to copy an image from one file to another using byte streams and handle FileNotFoundException and IOException using try-catch-finally.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.FileNotFoundException;
import java.io.IOException;

public class SimpleImageCopy {
    public static void main(String[] args) {
        FileInputStream fis = null;
        FileOutputStream fos = null;

        try {
            fis = new FileInputStream("source.jpg.jpeg");
            fos = new FileOutputStream("destination.jpg");

            byte[] buffer = new byte[1024];
            int length;
            while ((length = fis.read(buffer)) != -1) {
                fos.write(buffer, 0, length);
            }

            System.out.println("Image copied successfully.");

        } catch (FileNotFoundException e) {
            System.out.println("File not found.");
        } catch (IOException e) {
            System.out.println("Error copying file.");
        } finally {
            try {
                if (fis != null) fis.close();
                if (fos != null) fos.close();
            } catch (IOException e) {
                System.out.println("Error closing files.");
            }
        }
    }
}
```

```
C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>javac Sim
pleImageCopy.java

C:\Users\mpraz\OneDrive\Attachments\Desktop\JAVA LAB\java programs>java File
Copy
File copied successfully.
```